

Proposed System Upgrades: Scale, Security, and DX Improvements

This document lists recommended architectural upgrades to elevate the real-time Student Portal integration. These focus on scalability, application hygiene, vector search performance, and enhanced user experiences.

1. Enterprise Task Queueing: Celery + Redis

Current Design

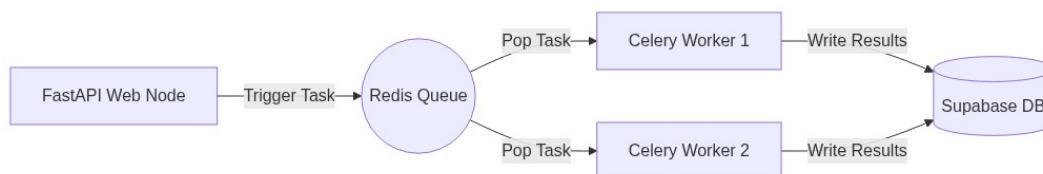
FastAPI BackgroundTasks run asynchronously in the main API process memory space.

The Risk

If the server restarts or encounters a peak load of hundreds of simultaneous student applications, in-memory tasks can be lost or block the main event loop.

Recommended Upgrade

Offload heavy AI computations (Sentence-Transformers and NVIDIA NIM racing) to Celery Workers backed by Redis as a message broker.



- Benefit: Ensures task persistence, enables horizontal scaling of AI worker nodes separate from web nodes, and prevents main API slowdowns.
-

2. HNSW Indexes for Vector Search Optimization

Current Design

Currently, semantic matching performs a linear scan (sequential scan) to calculate cosine similarity on vector embeddings.

The Risk

When the database grows to thousands of resumes and jobs, calculations will slow down significantly, leading to higher CPU consumption.

Recommended Upgrade

Configure Hierarchical Navigable Small World (HNSW) indexing in PostgreSQL for the pgvector columns.

```
-- Create HNSW indexes on embedding columns for cosine distance
CREATE INDEX IF NOT EXISTS idx_resumes_embedding_hnsw
ON public.resumes
USING hnsw (embedding vector_cosine_ops);

CREATE INDEX IF NOT EXISTS idx_job_descriptions_embedding_hnsw
ON public.job_descriptions
USING hnsw (embedding vector_cosine_ops);
```

- Benefit: Reduces query execution time from $O(N)$ to $O(\log N)$, ensuring semantic comparisons remain fast at scale.

3. Application Deduplication & Rate Limiting

The Risk

Students might spam the application endpoint with multiple uploads or request loops, inflating NVIDIA NIM LLM usage and inflating database storage.

Recommended Upgrades

1. Application Constraints: Add a unique composite index to the applications table to prevent duplicate applications for the same job.

```
CREATE UNIQUE INDEX IF NOT EXISTS uq_student_job_application
ON public.applications(student_id, job_id)
WHERE status != 'rejected';
```

2. Recruiter Status Enforcement: If a student is rejected, allow them to reapply only after a specified cooldown period (e.g., 90 days).
3. Payload Rate Limiter: Limit student resume uploads to a maximum of 5 uploads per hour per student IP/Auth session using backend middleware.

4. Fine-Grained Row Level Security (RLS) Policies

The Risk

Without strict RLS, a student could potentially access other candidates' resumes or edit match metrics by manipulating database connection calls.

Recommended Upgrade

Enable Supabase RLS and configure policies to lock down tables, ensuring that auth users only access their own slices of data:

```
ALTER TABLE public.resumes ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.applications ENABLE ROW LEVEL SECURITY;

-- Students can read/write their own applications
```

```

CREATE POLICY "Students manage own applications"
ON public.applications
FOR ALL
USING (auth.uid() = student_id);

-- Recruiters can read applications for jobs they own
CREATE POLICY "Recruiters read applications for their jobs"
ON public.applications
FOR SELECT
USING (
  EXISTS (
    SELECT 1 FROM public.job_descriptions jd
    WHERE jd.id = applications.job_id
    AND jd.user_id = auth.uid()::text
  )
);

```

5. Student Real-time Feedback Loops

Recommended Upgrades

1. Pre-flight ATS Check: Before submitting, allow students to run a dry-run ATS score on their resume draft. The system can offer suggestions (using the rewrite service engine) to align their resume text with the job description *prior* to formal submission.
2. Dynamic Stage Notifications: Implement real-time push alerts or status banners inside the Student Portal. When the recruiter changes the application status on their pipeline view (applied shortlisted / rejected), the student dashboard updates immediately:

```

// Student Portal Subscription to Status Changes
supabase
  .channel('my-applications')
  .on(
    'postgres_changes',
    { event: 'UPDATE', schema: 'public', table: 'applications', filter:
`student_id=eq.${studentId}` },
    (payload) => {
      showNotification(`Application status updated to: ${payload.new.status}`);
    }
  )
  .subscribe();

```